Substitution Method Analysis

CSA0606-Design and Analysis of Algorithms

Guided By,
Dr. K. Rajagopal

Presented By,
Aadhithya V
(192511256)

Introduction

What is the Substitution Method?

- Mathematical technique for solving recurrence relations
- Also known as the 'guess and verify' method
- Fundamental for analyzing algorithm complexity
- Works by substituting the recurrence into itself repeatedly
- Helps identify patterns and derive closed-form solutions

Problem Statement

Recurrence Relation

$$T(n) = T(n-1) + n$$

Base Case

- $T(1) = 1$

Objective

- Derive the closed-form solution and determine the time complexity

- **The given recurrence relation is $T(n) = T(n - 1) + n$**, with the base case **$T(1) = 1$**.
- **The objective is to solve the recurrence** using the **Substitution Method**.
- **The recurrence is expanded step by step** until the base case is reached.
- **A closed-form solution is derived** to represent the recurrence without recursion.
- **The final goal is to determine the time complexity** of the recurrence relation after obtaining the closed-form solution.

Algorithm / Pseudocode

Substitution Method Approach

```
Algorithm: Substitution Method
──────────────────────────────────

Step 1: Write the recurrence relation
        T(n) = T(n-1) + n

Step 2: Substitute T(n-1) recursively
        T(n) = T(n-2) + (n-1) + n

Step 3: Continue substitution k times
        T(n) = T(n-k) + (n-k+1) + ... + (n-1) + n

Step 4: Expand until base case is reached
        When n-k = 1, we have performed (n-1) substitutions

Step 5: Use summation formula
        Sum of i from 1 to n = n(n+1)/2

Step 6: Derive closed form and identify complexity
```

Step-by-Step Expansion

Substitution Method - Detailed Analysis

Step 1 $T(n) = T(n-1) + n$

Step 2 $= [T(n-2) + (n-1)] + n = T(n-2) + (n-1) + n$

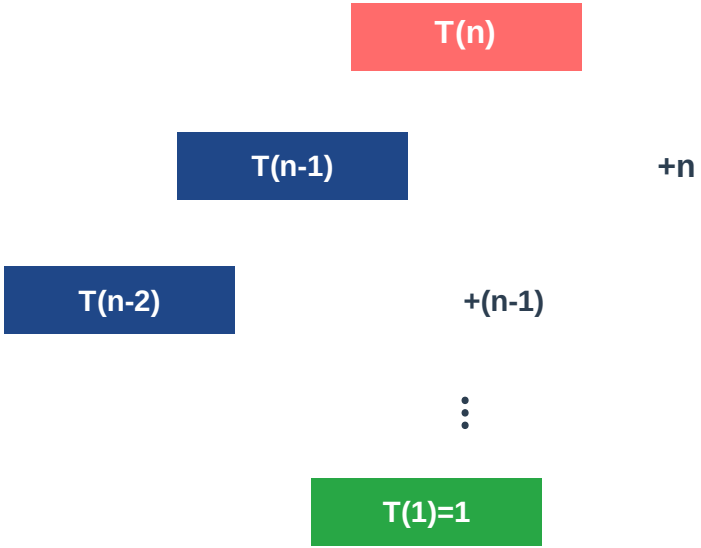
Step 3 $= T(n-3) + (n-2) + (n-1) + n$

Step 4 $= T(n-k) + (n-k+1) + \ldots + (n-1) + n$

Base Case: When $n-k = 1 \Longrightarrow k = n-1$
$T(1) = 1$, so after $(n-1)$ substitutions, we reach the base case

Architecture Diagram

Recursive Decomposition Tree



Depth = n-1 levels | Cost at each level increases

- **The diagram starts with T(n)**, which represents the original problem.
- **Each step calls T(n-1)**, reducing the problem size by 1.
- **At every level, the current value (+n, +(n-1), etc.) is added** to the result.
- **The recursion stops at T(1) = 1**, which is the base case.
- **The diagram shows how all the values are added together** to get the final answer **T(n) = n(n+1)/2**.

Implementation (Code)

Python Implementation

```
# Recursive Implementation
def T_recursive(n):
    if n == 1:
        return 1
    return T_recursive(n-1) + n

# Closed-form Solution
def T_closed_form(n):
    return n * (n + 1) // 2

# Example Usage
for n in range(1, 8):
    recursive = T_recursive(n)
    closed = T_closed_form(n)
    print(f"T({n}) = {recursive}")
```

- **Recursive Function** T_recursive(n) solves the recurrence relation **T(n) = T(n-1) + n** by calling itself until the base case is reached.
- **Base Case** if n == 1: return 1
- Stops the recursion and provides the initial value **T(1) = 1**.
- **Recursive Step** return T_recursive(n-1) + n
- Computes the previous result and adds the current value of **n**, building the sum step by step.
- **Closed-Form Solution** T_closed_form(n) uses the formula **n(n+1)/2** to calculate the result directly without recursion, making it more efficient.
- **Output Verification** The for loop runs from **n = 1 to 7**, computes the result using both recursive and closed-form methods, and verifies that both produce the same values.

Mathematical Derivation

From Substitution to Closed Form

After (n-1) substitutions with base case T(1) = 1:

$$T(n) = T(1) + 2 + 3 + \ldots + (n-1) + n$$

$$T(n) = 1 + \sum_{i=2}^{n} i$$

Using the formula: $\sum_{i=1}^{n} i = n(n+1)/2$

$$T(n) = \sum_{i=1}^{n} i = n(n+1)/2$$

Closed Form: $T(n) = n(n+1)/2 = (n^2 + n)/2$

Output Analysis

Computed Values for Various n

n	T(n) = n(n+1)/2	Verification
1	1	1×2/2 = 1
2	3	2×3/2 = 3
3	6	3×4/2 = 6
5	15	5×6/2 = 15
10	55	10×11/2 = 55
100	5050	100×101/2 = 5050

Complexity Analysis

Time & Space Complexity

Time Complexity

$O(n^2)$

Since $T(n) = n(n+1)/2 = (n^2 + n)/2$, the leading term is n^2, making this a quadratic complexity.

Space Complexity

$O(n)$

Recursive call stack depth is n due to (n-1) recursive calls before reaching the base case.

Efficiency: Linear time algorithms are preferred over quadratic ones. For large n, $O(n^2)$ becomes computationally expensive.

Conclusion

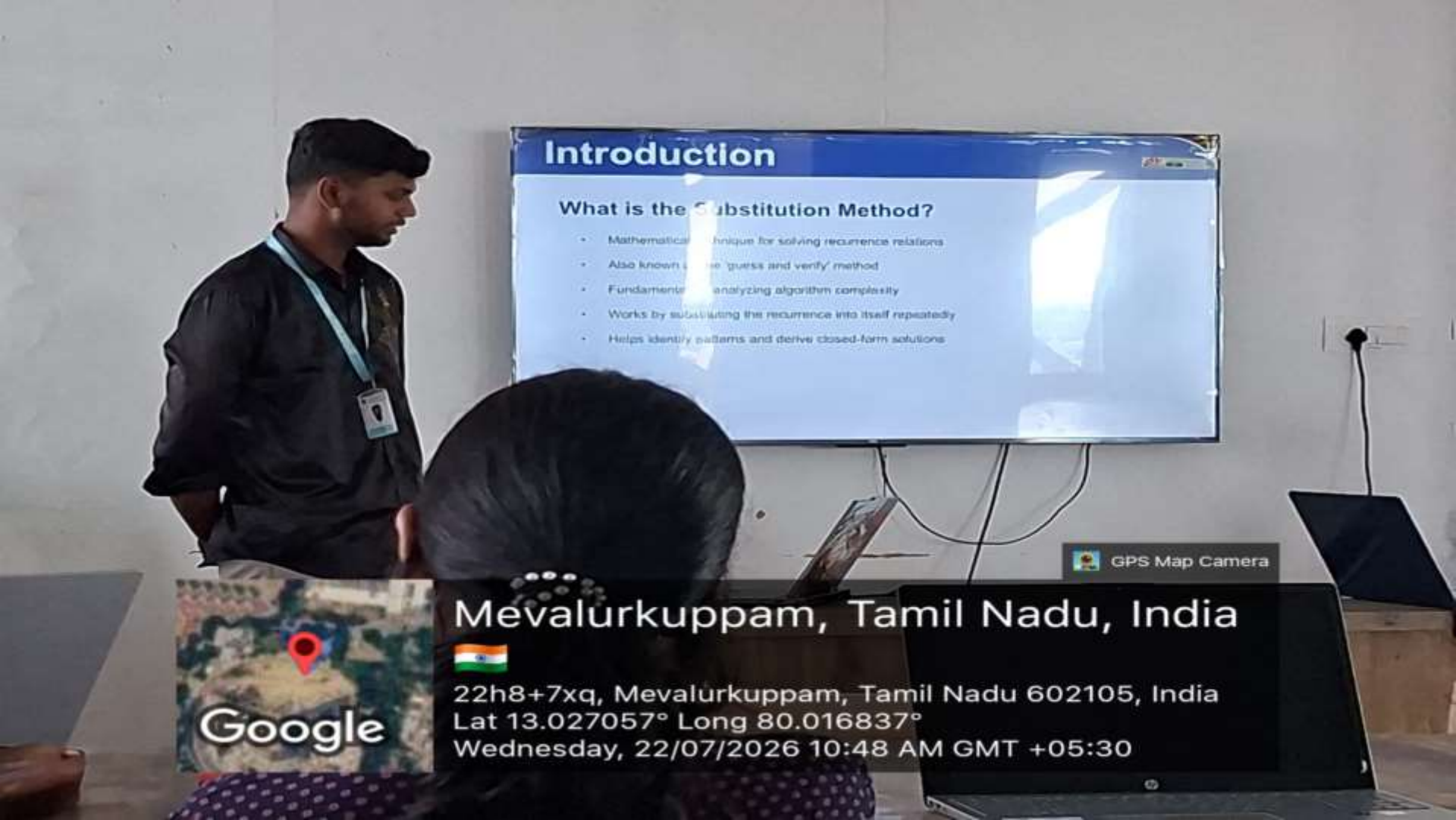
- The Substitution Method is a powerful technique for solving recurrence relations
- Through step-by-step expansion, we derived the closed form: $T(n) = n(n+1)/2$
- The algorithm exhibits quadratic time complexity $O(n^2)$
- Space complexity of $O(n)$ is due to recursive call stack
- This analysis demonstrates why iterative solutions are preferred in practice
- Understanding complexity helps in algorithm selection and optimization

References

Key Resources

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- [2] Kleinberg, J., & Tardos, E. (2005). Algorithm Design. Pearson Education.
- [3] Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
- [4] Rosen, K. H. (2011). Discrete Mathematics and Its Applications (7th ed.). McGraw-Hill.

Thank You!


Introduction
What is the Substitution Method?
Mathematical technique for solving recurrence relations
Also known as the 'guess and verify' method
Fundamental in analyzing algorithm complexity
Works by substituting the recurrence into itself repeatedly
Helps identify patterns and derive closed-form solutions
GPS Map Camera
Mevalurkuppam, Tamil Nadu, India
22h8+7xq, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.027057° Long 80.016837°
Wednesday, 22/07/2026 10:48 AM GMT +05:30
Google